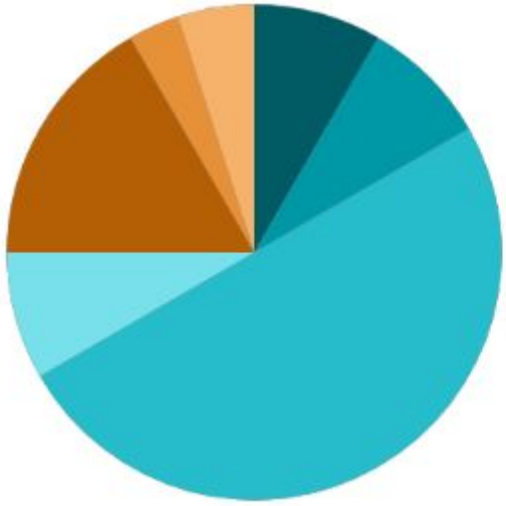
Multimedia

Dr. Bhoomi Shah & Ms. Hetal Jethani

Department of Computer Science & Engineering

Lecture Structure



05 Minutes	Outline & Objectives
05 Minutes	Recap of Previous Lecture
30 Minutes	Delivery of Actual Content as per Lesson Plan
05 Minutes	Industry Relevance
10 Minutes	Interactive Activities
02 Minutes	Importance of Discipline
03 Minutes	Recording of Attendance

Outline

1. Multimedia Handling
2. Data Storage Basics
3. Local Structured Data Storage
4. Data Sharing and Remote Storage

Objectives

1. To enable students to play and manage multimedia content.
2. To understand and implement different data storage options.
3. To develop the ability to create and manage local databases.
4. To integrate remote database services and cloud-based solutions.

Recap of Previous Lecture

- **Mobile communication** enables wireless data and voice exchange, offering portability, mobility, and real-time access through devices like smartphones and tablets.
- **Wireless technologies** have evolved from 1G (analog) to 5G (high-speed, low-latency), with GSM providing a structured framework for global communication.
- **Mobile computing systems** follow a 3-tier architecture—presentation (UI), application (logic), and data (storage)—ensuring efficient, scalable, and secure operation.
- **Ad-hoc and satellite networks** (MANETs and GMSS) support communication in remote, dynamic, or infrastructure-less environments, vital for military, disaster relief, and remote industries.
- **Mobile agents** are intelligent, self-moving programs that perform tasks independently across networks, improving system performance and reducing load.

Multimedia

- Multimedia (multi meaning many and media meaning means of storing and/or communication any information) is a type of medium which is created by using more than one conventional medium.
- It is any blend of text, images, audio, animation and video delivered and manipulated electronically. One example of multimedia could be the collection of text, images and animation in a Computer Based Training (CBT) for lessons in Geography.
- For a person surfing the World Wide Web, it can be web pages with a diverse collection of text, images, graphics, audio, animation,and embedded video. For a video game kiosk (like a Windows X station), it is a richly simulated environment inclusive of situational audio and video along with textual instructions.
- We come across multimedia elements in a number of situations. Our lives are so wrapped in digital media that we rarely take cognizance that a particular blend is a combination of a number of individual medium of information transfer.

Multimedia

- The various types of media which form multimedia are text, images, audio, animation, moving images, and video. Text is one of the oldest and prominent types of information carrier.
- Text delivers information which can have an effective meaning. Even to this day, most of the crucial data and information is maintained in textual form only.
- With the advent of Web, information in the textual format has had the effect of being exploded. The most popular messaging media, SMS is an example of text.

Multimedia

- Sound or audio is another form of medium very commonly included as a part of any multimedia presentation.
- It is one of the most sensual elements of any multimedia content, the effect of occurrence of which can be perceived over large distances, unlike other media.
- The greatest advantage of audio is that it does not depend on the literary skill of an individual. People who cannot read or write can use voice to communicate and express their thoughts.

Why Multimedia

- Multimedia is a new technological phenomenon proliferating on the advancements of faster processing and communication technologies; cheaper and physically shrinking storage technologies;and greater content creation, capability, and awareness.
- We enumerate the major benefits of using multimedia technologies in the following points:
- 1. Multiple and interactive media helps in easier grasp of any subject.
- 2. Digitization helps revolutionizing music, movies, games and virtual reality; most important of all, digitized objects do not need electro-mechanical devices for playing; like we do not need a turntable to play a record or a CD.
- 3. Digitized objects have other advantages that make it very attractive. It consumes less power,can take advantage of increasing processing power, is easy to compress, easy to store and archive. It can be edited and altered very easily.

Why Multimedia

- 4. There remains huge potential for new applications and services based on multimedia. This implies more benefit for markets and avenues for improving our lives (in terms of learning,work and leisure).
- 5. Smarter devices with lesser memory footprints, easy to use and understand interfaces offer quick network connectivity.
- 6. Digitization of nearly every form and every chunk of data.
- 7. Technologies for organizing, browsing, storing and retrieving multimedia content is improving at a very rapid rate. Also, the same storage can be used over and over again.

Major Application Arenas of Multimedia

- The application areas for multimedia content are many. Broadly, these can be classified as under:
- 1. Residential services such as Video on Demand (VoD), Home shopping, Conferencing, etc.
- 2. Business services including corporate learning, e-business, simulations, conferencing, etc.
- 3. Education such as distance learning, self learning, digital libraries, etc.
- 4. Scientific research such as scientific visualization, prototyping, simulations, medicines, etc.
- 5. Entertainment and leisure activities.
- 6. Web applications.

Playing Audio

- Types of Audio:

- Background Music: Continuous sound that plays while the app is in use, creating an immersive atmosphere.
- Sound Effects: Short audio clips that play in response to user actions, like button presses or notifications.
- Voiceovers: Narration that guides users through app content or functionality.

Playing Audio

- Android: The MediaPlayer class can be used to play audio.

```java
MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.sound);
mediaPlayer.start();
```

- Optimize audio file sizes to ensure quick loading and reduced memory usage.
- Manage audio sessions properly to prevent interruptions.

Playing Video

- ExoPlayer is a powerful library for playing video.
- User Interface Considerations:
- Ensure that users can easily control playback with buttons for play, pause, and rewind.
- Implement full-screen viewing options to enhance user experience.

```java
SimpleExoPlayer player = new SimpleExoPlayer.Builder(context).build();
PlayerView playerView = findViewById(R.id.player_view);
playerView.setPlayer(player);

MediaItem mediaItem = MediaItem.fromUri("https://example.com/video.mp4");
player.setMediaItem(mediaItem);
player.prepare();
player.play();
```

Rotate Animation

- Rotating elements on the screen can create dynamic visuals, drawing users' attention to specific parts of the app.
- Use Cases: Loading indicators, interactive graphics in games, or transitioning between screens.
- Android: Use ObjectAnimator for smooth rotation.

```java
ObjectAnimator rotation = ObjectAnimator.ofFloat(yourView, "rotation", 0f, 360f);
rotation.setDuration(1000);
rotation.setRepeatCount(ValueAnimator.INFINITE);
rotation.start();
```

Key Features of setOneShot()

The setOneShot(boolean oneShot) method in Android's frame animation controls whether the animation should loop or play only once. It's a part of the AnimationDrawable class, which is used for creating frame-by-frame animations in Android.

Key Features of setOneShot()

1. **Parameter**: The method takes a boolean value:
   - true: The animation will play only once.
   - false: The animation will loop continuously.
2. **Default Behavior**: By default, frame animations loop indefinitely. If you want the animation to stop after one cycle, you should call setOneShot(true).

Key Features of setOneShot()

Here's how to use setOneShot() in an Android application:

1. **Creating Frame Animation**: First, define your animation frames in XML.

```xml
<!-- res/drawable/animation_list.xml -->
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false"> <!-- Default is false, but can be changed in code -->
    <item android:drawable="@drawable/frame1" android:duration="100" />
    <item android:drawable="@drawable/frame2" android:duration="100" />
    <item android:drawable="@drawable/frame3" android:duration="100" />
</animation-list>
```

Key Features of setOneShot()

2.Applying the Animation:

In your activity or fragment, you can load and start the animation. Here's how to set it to play only once:

```java
AnimationDrawable animationDrawable = (AnimationDrawable) imageView.getDrawable();
animationDrawable.setOneShot(true); // Set to play only once
animationDrawable.start();
```

Use Cases

- **Non-repeating Animations**: Useful for scenarios where you want a brief animation effect, such as indicating a state change (e.g., a loading spinner that animates once).
- **Interactive Feedback**: Great for providing visual feedback on user actions, like button presses or transitions.

Fade In/Fade Out Animation

- Definition: This animation gradually changes an element's opacity, enhancing visual transitions in your app.
- Use Cases: Alerts, modal views, or transitioning between different sections of the app.
- **Fade In:** This will gradually change the visibility of a view from invisible to visible.
- **Fade Out:** This will gradually change the visibility of a view from visible to invisible.
- To create a simple fade-in and fade-out animation in an Android app, you can use Android's **ViewPropertyAnimator** or **ObjectAnimator**

Fade In/Fade Out Animation

xml Copy code

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, World!"
        android:textSize="24sp"
        android:visibility="gone"
        android:alpha="0" />
```

Fade In/Fade Out Animation

```
    <Button
        android:id="@+id/fadeInButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fade In"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/fadeOutButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fade Out"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

Fade In/Fade Out Animation

```java
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private TextView textView;
    private Button fadeInButton, fadeOutButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        textView = findViewById(R.id.textView);
        fadeInButton = findViewById(R.id.fadeInButton);
        fadeOutButton = findViewById(R.i adeOutButton);
```

Fade In/Fade Out Animation

```
        fadeInButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                textView.setVisibility(View.VISIBLE);  // Make it visible
                textView.animate().alpha(1.0f).setDuration(500);  // Fade in animation
            }
        });

        fadeOutButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                textView.animate().alpha(0.0f).setDuration(500).withEndAction(new Runnable
                    @Override
                    public void run() {
                        textView.setVisibility(View.GONE);  // Hide after fading out
                    }
                });
            }
        });
    }
}
```

Zoom Animation

- Definition: This animation allows users to scale elements, typically through pinch gestures, for an enhanced interactive experience.
- Use Cases: Image galleries, map views, or product details.

xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Zoom Me!"
        android:textSize="24sp" />
```

Zoom Animation

```
    <Button
        android:id="@+id/zoomInButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Zoom In"
        android:layout_marginTop="20dp"/>

    <Button
        android:id="@+id/zoomOutButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Zoom Out"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

Zoom Animation

java

```java
import android.os.Bundle;
import android.view.View;
import android.view.animation.ScaleAnimation;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private TextView textView;
    private Button zoomInButton, zoomOutButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

Zoom Animation

```
textView = findViewById(R.id.textView);
zoomInButton = findViewById(R.id.zoomInButton);
zoomOutButton = findViewById(R.id.zoomOutButton);

zoomInButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        ScaleAnimation zoomIn = new ScaleAnimation(
            1.0f, 1.5f,  // Start and end values for the X axis (1.0 is the origin
            1.0f, 1.5f,  // Start and end values for the Y axis
            ScaleAnimation.RELATIVE_TO_SELF, 0.5f,  // Pivot point for X (center o
            ScaleAnimation.RELATIVE_TO_SELF, 0.5f); // Pivot point for Y (center o
        zoomIn.setDuration(500);  // Duration for the animation
        zoomIn.setFillAfter(true);  // Maintain the final size after the animation
        textView.startAnimation(zoomIn);
    }
});
```

Zoom Animation

```
        zoomOutButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                ScaleAnimation zoomOut = new ScaleAnimation(
                    1.5f, 1.0f,  // Start and end values for the X axis
                    1.5f, 1.0f,  // Start and end values for the Y axis
                    ScaleAnimation.RELATIVE_TO_SELF, 0.5f,  // Pivot point for X (center o
                    ScaleAnimation.RELATIVE_TO_SELF, 0.5f); // Pivot point for Y (center o
                zoomOut.setDuration(500);  // Duration for the animation
                zoomOut.setFillAfter(true);  // Maintain the final size after the animatio
                textView.startAnimation(zoomOut);
            }
        });
    }
}
```

Scale Animation

- Definition: Dynamically resizing UI elements in response to user interactions, enhancing the feedback loop.
- Use Cases: Interactive buttons, icons, or other UI components that need emphasis.

```java
yourView.setAlpha(0f); // Start completely transparent
yourView.animate().alpha(1f).setDuration(1000); // Fade in
```

2D Graphics

- Importance: 2D graphics are crucial for creating visually appealing interfaces and interactions.
- **Tools and Libraries:**
- Android: Canvas API provides a rich set of tools for drawing shapes and images.
- **Techniques:**
- Use vector graphics for scalability without losing quality.
- Utilize sprite sheets for smooth animations by combining multiple frames into a single image.

3D Graphics

- Definition and Use Cases: 3D graphics enhance realism and depth, ideal for applications like games and augmented reality (AR).
- Tools and Libraries:
- Android: OpenGL ES allows for low-level graphics rendering.

```java
ScaleGestureDetector scaleGestureDetector = new ScaleGestureDetector(context, new

private class ScaleListener extends ScaleGestureDetector.SimpleOnScaleGestureListe
    @Override
    public boolean onScale(ScaleGestureDetector detector) {
        yourView.setScaleX(yourView.getScaleX() * detector.getScaleFactor());
        yourView.setScaleY(yourView.getScaleY() * detector.getScaleFactor());
        return true;
    }
}
```

Conclusion

- Incorporating multimedia in Android applications is not just a trend; it significantly enhances functionality, user engagement, and overall satisfaction. As user expectations continue to rise, utilizing multimedia effectively will be crucial for app developers aiming for success in the competitive mobile landscape.

Data Storage

- Data storage is a crucial aspect of mobile application development, allowing apps to persist data even after they are closed or the device is restarted. In Android development using Java, various storage options cater to different use cases, ranging from simple key-value pairs to complex databases.

Shared Preferences

- Shared Preferences is a lightweight mechanism for storing small amounts of primitive data (key-value pairs). It is typically used for saving user preferences or application settings.
- Use Cases:
  - Storing user settings (e.g., theme, language)
  - Managing user login states (e.g., remember me functionality)
- **Key Features of Shared Preferences**

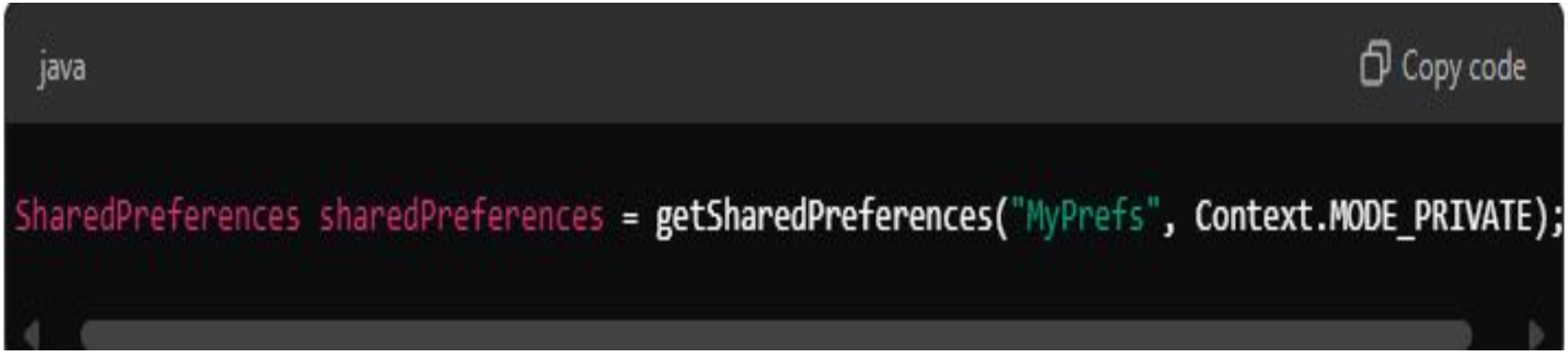
1. **Simple Storage**: Ideal for storing simple data like user settings, preferences, or app state (e.g., theme settings, user login status).
2. **Key-Value Pairs**: Data is stored as key-value pairs, allowing easy retrieval using the associated key.
3. **Persistence**: Data persists across user sessions, making it suitable for storing information that needs to be retained.
4. **Lightweight**: It is a lightweight storage solution for small datasets.

How to use Shared Preferences

Accessing Shared Preferences

You can access Shared Preferences in two ways:

- **Using getSharedPreferences()**: For multiple preference files.
- **Using getPreferences()**: For a single preference file associated with the activity.



Shared Preferences

Storing Data

To store data, you need to get an Editor object from the SharedPreferences instance. You can then use methods like putString(), putInt(), etc., to store different data types.

```java
SharedPreferences.Editor editor = sharedPreferences.edit();
editor.putString("username", "JohnDoe");
editor.putInt("userAge", 30);
editor.putBoolean("isLoggedIn", true);
editor.apply(); // or editor.commit();
```

- **apply()**: Saves changes in the background without blocking the UI thread.
- **commit()**: Saves changes synchronously and returns a boolean indicating success.

Shared Preferences

java Copy code

```java
ObjectAnimator scaleX = ObjectAnimator.ofFloat(yourView, "scaleX", 1f, 1.5f);
ObjectAnimator scaleY = ObjectAnimator.ofFloat(yourView, "scaleY", 1f, 1.5f);
AnimatorSet scaleAnimator = new AnimatorSet();
scaleAnimator.play(scaleX).with(scaleY);
scaleAnimator.setDuration(500);
scaleAnimator.start();
```

Internal Storage

- Internal storage is a private storage area for an app. Files saved here are not accessible to other applications, making it suitable for sensitive data.
- Use Cases:
  - Storing configuration files, user credentials, or any sensitive information.

java Copy code

```java
public class MyGLRenderer implements GLSurfaceView.Renderer {
    @Override
    public void onDrawFrame(GL10 gl) {
        // Custom rendering logic here
    }
    // Implement other necessary methods for context, size, etc.
}
```

Internal Storage

java Copy code

```java
// Save data
SharedPreferences sharedPreferences = getSharedPreferences("MyPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPreferences.edit();
editor.putString("username", "user123");
editor.putBoolean("isLoggedIn", true);
editor.apply(); // Asynchronous save

// Retrieve data
SharedPreferences sharedPreferences = getSharedPreferences("MyPrefs", MODE_PRIVATE);
String username = sharedPreferences.getString("username", "defaultUser");
boolean isLoggedIn = sharedPreferences.getBoolean("isLoggedIn", false);
```

External Storage

External storage is a shared storage space that allows you to save files that other apps can access. This can be an SD card or internal storage that is not private to your app.

```
// Writing to internal storage
String filename = "myfile.txt";
String fileContents = "Hello World!";
FileOutputStream fos = null;
try {
    fos = openFileOutput(filename, MODE_PRIVATE);
    fos.write(fileContents.getBytes());
} catch (Exception e) {
    e.printStackTrace();
} finally {
    if (fos != null) {
        try {
            fos.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

External Storage

```
// Reading from internal storage
FileInputStream fis = null;
try {
    fis = openFileInput(filename);
    InputStreamReader isr = new InputStreamReader(fis);
    BufferedReader br = new BufferedReader(isr);
    StringBuilder sb = new StringBuilder();
    String line;
    while ((line = br.readLine()) != null) {
        sb.append(line);
    }
    String fileContent = sb.toString();
} catch (Exception e) {
    e.printStackTrace();
} finally {
    if (fis != null) {
        try {
            fis.close();
        } catch (IOException e) {
            e.printStackTrace();
```

SQLite Databases

SQLite is a lightweight database engine that's built into Android. It's suitable for storing structured data.

- SQLite is a lightweight, serverless database engine embedded within Android, making it an ideal choice for local data storage in mobile applications.
- It provides a relational database structure, allowing developers to efficiently manage structured data with the ability to execute complex queries using SQL.
- Each application has its own private database, ensuring data security and integrity. SQLite supports transactions, enabling safe and atomic operations on data. Its compact size and high performance make it suitable for various applications, from simple data logging to more complex data management scenarios.
- Developers can easily interact with SQLite databases through the Android SQLite API, utilizing classes like SQLiteOpenHelper for managing database creation and versioning, and SQLiteDatabase for executing SQL commands.
- This integration allows for robust data handling while maintaining the lightweight nature of mobile applications.

SQLite Databases

- SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured SQL database engine. It is integrated into Android, making it easy to work with without needing to include external dependencies.

Key Features:

- **Serverless Architecture:** SQLite does not run as a separate server. Instead, the database is stored as a single file on disk, and applications access this file directly.
- **ACID-compliant:** SQLite supports Atomicity, Consistency, Isolation, and Durability. This ensures that all database transactions are processed reliably.
- **Cross-Platform:** SQLite is cross-platform, meaning the database file can be moved between Android, iOS, and other platforms without needing to convert it.
- **No Setup Required:** Unlike traditional databases that need a server, SQLite is embedded and ready to use without any configuration.

Why Use SQLite in Android?

- **Local Storage:** SQLite allows Android applications to store data locally on the device, making it accessible even when offline.
- **Relational Database:** It supports SQL queries, transactions, and ACID (Atomicity, Consistency, Isolation, Durability) compliance, providing structured and reliable data management.
- **Lightweight:** SQLite doesn't require a server or configuration, making it ideal for resource-constrained environments like mobile apps.
- **Integrated with Android:** Android has native support for SQLite, making it easy to use without needing external libraries.

Key Concepts of SQLite in Android

- **SQLiteOpenHelper:** This is a helper class used to manage database creation and version management. You can override onCreate() and onUpgrade() methods to define the behavior when the database is created or upgraded.
- **Cursor:** Cursors are used to iterate over the rows returned by a query. They provide methods to navigate through the data and retrieve individual values.
- **ContentValues:** This class allows you to store a set of values that the insert() method uses to insert rows into a table.
- **SQL Queries:** You can perform standard SQL operations like SELECT, INSERT, UPDATE, and DELETE using SQLite in Android.

- **Dependencies**
- SQLite comes built into Android, so no external libraries are required. However, for more complex scenarios, you might consider using libraries like Room, which provides an abstraction layer over SQLite.

Creating a Database

To create an SQLite database, you typically extend the **SQLiteOpenHelper** class, which helps manage database creation and version management.

```
public class DatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "myDatabase.db";
    private static final int DATABASE_VERSION = 1;

    public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createTable = "CREATE TABLE myTable (id INTEGER PRIMARY KEY AUTOINCREMENT,
        db.execSQL(createTable);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS myTable");
        onCreate(db);
    }
```

Key Features of execSQL(String sql)

The execSQL(String sql) method in SQLite is used to execute a single SQL statement that doesn't return any data, such as data manipulation or schema modification commands. This method is part of the SQLiteDatabase class in Android and is essential for performing operations like creating tables, inserting data, updating data, and deleting records.

- **No Return Value**: Since this method is designed for SQL statements that don't produce a result set, it does not return any data. Instead, it returns void.
- **SQL Commands**: You can use execSQL() for a variety of SQL commands:
- **DDL (Data Definition Language)**: Commands like CREATE TABLE, DROP TABLE, and ALTER TABLE.
- **DML (Data Manipulation Language)**: Commands like INSERT, UPDATE, and DELETE.
- **Prepared Statements**: This method does not support parameterized queries directly. For operations that require parameters, you should use methods like insert(), update(), or delete(), or use SQLiteStatement.

Performing CRUD Operations

Creating Records

To insert data, you can use the insert() method:

```java
ContentValues values = new ContentValues();
values.put("name", "John Doe");
SQLiteDatabase db = this.getWritableDatabase();
db.insert("myTable", null, values);
db.close();
```

SQLite Databases

Reading Records

To retrieve data, you can use the query() method:

```java
SQLiteDatabase db = this.getReadableDatabase();
Cursor cursor = db.query("myTable", null, null, null, null, null, null);
if (cursor.moveToFirst()) {
    do {
        String name = cursor.getString(cursor.getColumnIndex("name"));
    } while (cursor.moveToNext());
}
cursor.close();
db.close();
```

SQLite Databases

Updating Records

To update data, use the update() method

```java
ContentValues values = new ContentValues();
values.put("name", "Jane Doe");
SQLiteDatabase db = this.getWritableDatabase();
db.update("myTable", values, "id = ?", new String[]{"1"});
db.close();
```

SQLite Databases

Deleting Records

To delete data, use the delete() method:

```java
SQLiteDatabase db = this.getWritableDatabase();
db.delete("myTable", "id = ?", new String[]{"1"});
db.close();
```

Conclusion

SQLite is a powerful tool for data storage in Android applications, offering a balance between simplicity and functionality. Whether using raw SQLite or leveraging Room for a more structured approach, understanding how to implement and manage SQLite databases effectively is crucial for building robust Android applications.

Content Provider

- In Android, a Content Provider is a key component used to manage access to a structured set of data and to share data between different apps securely.
- Additionally, Remote Databases are commonly used for syncing data between an Android device and a server, enabling cloud storage or shared data access across multiple devices.
- A Content Provider in Android offers a consistent interface to access data that is stored either in the app itself (such as in an SQLite database) or in some external storage (such as contacts, media files, etc.).
- It is primarily used to share data between different applications, providing a mechanism to read, write, and update data, while enforcing security permissions.
- **URI (Uniform Resource Identifier):** A Content URI is the identifier for the data managed by a content provider. The URI usually starts with content://, followed by the authority (the provider's identifier) and the path to the data.

Example: content://com.example.myapp.provider/users/1

Implementing a Content Provider

A custom content provider is created by subclassing the ContentProvider class and implementing the following methods:

- onCreate(): Initializes the provider. Called when the provider starts.
- query(): Handles query requests (read operations).
- insert(): Handles requests to insert new data.
- update(): Handles update requests.
- delete(): Handles requests to delete data.
- getType(): Returns the MIME type for a URI.

Implementing a Content Provider

```
public class UserContentProvider extends ContentProvider {

    private static final String AUTHORITY = "com.example.myapp.provider";
    private static final String USERS_TABLE = "users";
    public static final Uri CONTENT_URI = Uri.parse("content://" + AUTHORITY + "/" + USERS

    private static final int USERS = 1;
    private static final int USER_ID = 2;

    private static final UriMatcher uriMatcher;
    static {
        uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
        uriMatcher.addURI(AUTHORITY, USERS_TABLE, USERS);
        uriMatcher.addURI(AUTHORITY, USERS_TABLE + "/#", USER_ID);
    }

    private SQLiteDatabase database;
```

Using a Content Provider

In an app, you can use a ContentResolver to interact with a content provider by calling methods like query(), insert(), update(), and delete().

Query Example:

```java
Uri uri = Uri.parse("content://com.example.myapp.provider/users");
Cursor cursor = getContentResolver().query(uri, null, null, null, null);
```

Insert Example:

```java
ContentValues values = new ContentValues();
values.put("name", "John Doe");
values.put("email", "john@example.com");
Uri newUri = getContentResolver().insert(uri, values);
```

Remote Databases

In Android mobile application development, a remote database refers to a database hosted on a remote server, typically in the cloud, which the mobile app accesses over the internet. This allows the application to store, retrieve, and manage data centrally, providing features like data synchronization, scalability, multi-device access, and persistence across sessions.

Why Use Remote Databases?

Remote databases are critical for applications that require:
Data synchronization across multiple devices.
Large-scale data management that would be impractical to store locally on the device.
Collaboration between users or apps that share the same data (e.g., social media apps, chat applications).
Centralized data management, enabling cloud storage and backup solutions.
Real-time updates, like in gaming, messaging, or e-commerce applications.
Examples of apps that rely heavily on remote databases include WhatsApp (messages synced across devices), Instagram (media stored in the cloud), and Gmail (email data accessed from servers).

Common Remote Database Solutions for Android Apps

Several options are available for using remote databases in Android development:

- MySQL/PostgreSQL with RESTful APIs
- Firebase Realtime Database
- Firebase Firestore
- Amazon Web Services (AWS)
- Microsoft Azure Cosmos DB
- MongoDB Atlas
- Supabase

Industry Relevance

How API,Activities,Layouts and Widgets useful for the industry:

Companies like **Zomato, Ola, and Paytm** rely on Android APIs to access location, send notifications, etc.

Activities are used in banking apps, fitness trackers, and educational platforms.

Widgets are the **building blocks of UI/UX** in Android applications.

Industry Relevance Continued....

How Mobile Computing is useful for the industry:

Summary

Component	Industry Relevance
API	Enables complex, real-world functionality
Activity	Controls app flow and user interaction
Layout	Delivers professional, responsive design
Widget	Enhances interactivity and usability

Interactive Activity-Mobile Computing :

A real-world scenario to design, such as:

A **login screen**

A **food ordering app**

A **fitness tracking dashboard**

A **weather app home screen**

GSFCU & Values : Cultivating a Positive Mindset

Recommended Books